# AirSafe Device: Simulation and Software Test Guide

*Wokwi + VS Code · based on your AirSafe Complete Circuit Connection Diagram (Without Buzzer) · 7 Oct 2026*

## 1. Goal and approach

Run your real ESP32 firmware in the Wokwi simulator inside VS Code, feed it simulated sensors, stream the data into your software, and prove the whole chain before buying parts: device → transport → backend → AI/analysis → dashboard → relay command back to the device.

Three rules keep the test honest:

1. Simulate in layers: core board first, then stand-in sensors, then faithful sensor emulation.
2. Use one firmware and one data format for simulation and real hardware. Only the data source changes.
3. Treat simulation as proof of logic and protocol, not of electrical behaviour (Section 12 lists what it cannot prove).

> I read your diagram from a screenshot. The block diagram was clear, but the small pin-summary table was not fully legible. Confirm every pin number in Section 3 against your original file.

## 2. Circuit review

I checked the diagram block by block. Items 1 to 3 need a decision before you build. The rest are checks.

**1. Power path (check first).** The diagram shows AC-DC 5 V → TP4056 → Li-ion 3.7 V → buck converter labelled 5 V to 3.3 V. A TP4056 outputs battery voltage (about 3.0 to 4.2 V), not 5 V. A step-down converter cannot hold 3.3 V from a cell that sags towards 3.0 V, and nothing in the diagram creates a 5 V rail when running on battery, yet the MH-Z19B, PMS5003, relay and a mmWave occupancy sensor need 5 V. Pick one fix:

- *No battery backup:* run everything from the 5 V supply and derive 3.3 V from it with a buck converter or LDO.
- *Keep the battery:* battery → boost converter → 5 V rail → 3.3 V regulator. Use a charger board with protection (DW01) or, better, a power-path charger that can run the load while charging.

**2. Three UART sensors, two free UARTs.** MH-Z19B, PMS5003 and the occupancy sensor are all UART devices. The ESP32 has three hardware UARTs, but UART0 is used for flashing and serial logs. Options: (a) use a PIR or an I2C ToF sensor for occupancy; (b) keep the LD2410 on a hardware UART (it runs at 256000 baud, too fast for software serial) and read the MH-Z19B through software serial, its PWM output, or swap in an I2C CO2 sensor; (c) use UART0 for a sensor and give up USB logging (not recommended while developing).

**3. Occupancy sensor: ToF or LD2410?** The label names both, but they are different devices. The HLK-LD2410 is a mmWave radar on UART. A ToF module is an I2C distance sensor. A PIR needs one GPIO. Choose one. This guide covers all three.

**4. Analog air-velocity sensor.** Use an ADC1 pin (GPIO32 to 39); ADC2 pins stop working when WiFi is on. The ESP32 ADC tops out near 3.1 to 3.3 V, so if the sensor can output up to 5 V, add a divider (for example 10 kΩ top and 15 kΩ bottom gives 3.0 V from 5 V) and a 100 nF capacitor at the pin.

**5. ESP32 supply.** Power the dev board through one input only (VIN/5V or 3V3). Feeding both lets the onboard regulator fight your external supply. Confirm which pins your diagram's rails actually connect to.

**6. I2C bus is fine.** SHT31 0x44, SGP30 0x58, SSD1306 0x3C and the XGZP6847A (see its datasheet, commonly 0x6D) do not clash. Keep wires short. One set of 4.7 kΩ pull-ups is enough (breakouts often include them).

**7. Pins to avoid.** GPIO0, 2, 12 and 15 are boot-strapping pins, GPIO6 to 11 are the flash, and GPIO34 to 39 are input-only (fine for ADC and UART RX, not for outputs).

**8. Safety.** Use a certified, enclosed 5 V mains module with a fuse. If the relay switches mains loads (fan, damper), use a relay rated for it, keep mains wiring separated from the low-voltage board, and never test mains on a breadboard.

**9. Worth adding.** A battery-voltage divider (100 kΩ / 100 kΩ) to an ADC1 pin so the software can report battery level; the PMS5003 SET pin so the fan can be duty-cycled; per-sensor warm-up flags in firmware.

**Power budget.** Your table shows about 450 to 650 mA in total. At 5 V that is 2.2 to 3.3 W, so a 2600 mAh cell (about 9.6 Wh) gives roughly 2 to 3 hours after conversion losses.

## 3. Pin map and circuit diagrams

The pin map is proposed, based on the pins I could read in your diagram. Adjust it to match your original.

| Function | ESP32 pins | Notes |
|---|---|---|
| I2C bus (SHT31, SGP30, XGZP6847A, OLED) | SDA 21, SCL 22 | shared bus |
| microSD (SPI) | CS 5, SCK 18, MISO 19, MOSI 23 | |
| MH-Z19B (UART2) | ESP RX 17 ← sensor TX, ESP TX 16 → sensor RX | 9600 baud |
| PMS5003 (UART1, remapped) | ESP RX 32 ← sensor TX, TX 33 (optional) | 9600 baud, streams frames |
| Air velocity (analog) | GPIO34 (ADC1) | divider if output above 3.3 V |
| Occupancy | PIR or digital input on GPIO35, or LD2410 per item 2 | |
| RGB status LED | R 25, G 26, B 27 via resistors | |
| Relay IN | GPIO4 | check whether the module is active-low |
| Battery sense (optional) | GPIO36 (ADC1) | 100 kΩ / 100 kΩ divider |

The next two pages show the corrected circuit. Figure 1 is the signal wiring, with the pin map above applied. Figure 2 is the power section with the battery fix.

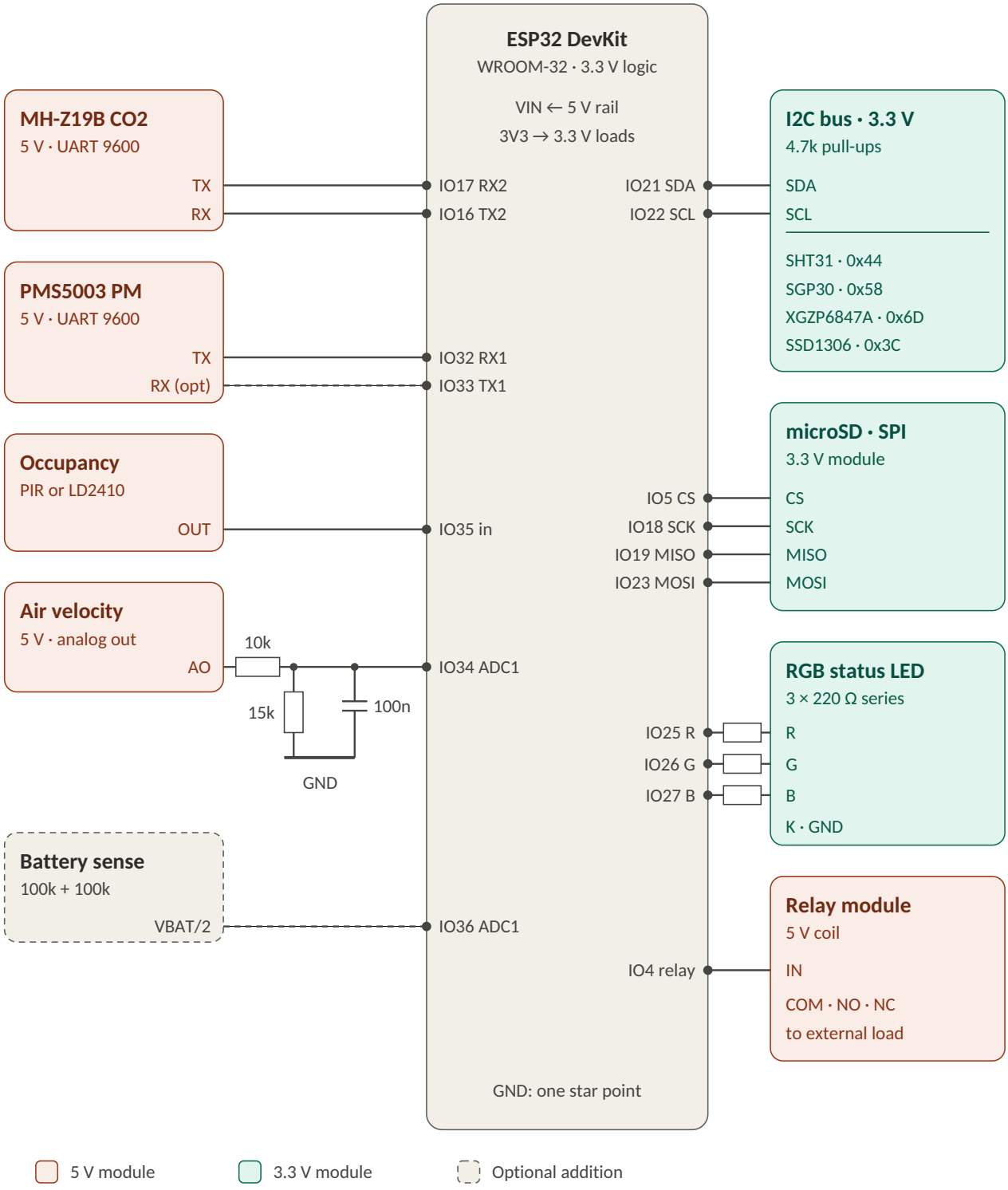



**Figure 1.** Signal wiring: ESP32 pins to every module. Coral = 5 V modules, teal = 3.3 V modules, dashed = optional.

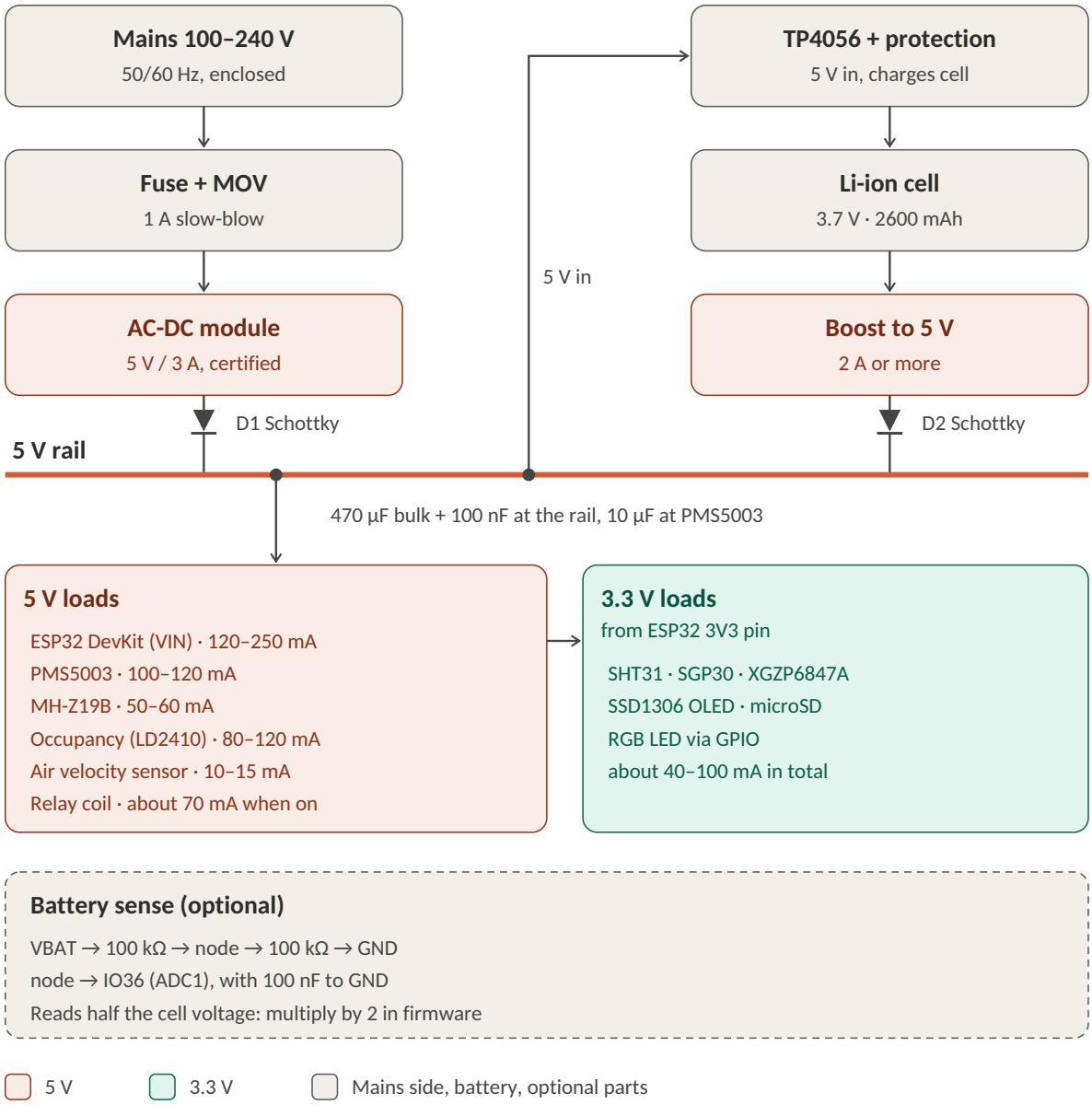



**Figure 2.** Corrected power section: mains and battery both feed one 5 V rail through diodes; the ESP32 3V3 pin feeds the 3.3 V loads.

Notes on the corrected circuit:

- Each module also needs VCC and GND, left out of Figure 1 for readability. The colour tells you which rail to use.
- The ESP32 board is fed through VIN from the 5 V rail, and its 3V3 pin supplies the low-current 3.3 V modules. If those loads grow beyond roughly 150 mA, add a separate 3.3 V buck converter, power the ESP32 through 3V3, and unplug USB while it runs.
- Occupancy is drawn as one digital line. An LD2410 needs a hardware UART instead (review item 2).
- Dashed lines and boxes are optional: the PMS5003 RX line (only needed to send it commands) and the battery sense input.
- Build the mains side only with a certified enclosed module and a fuse, and never test it on a breadboard.

## 4. Setup: VS Code + Wokwi (step by step)

1. Install VS Code, then the **PlatformIO IDE** and **Wokwi Simulator** extensions.
2. In VS Code run **Wokwi: Request a new License** and sign in at wokwi.com (a free community licence is available; check current terms).
3. Create a PlatformIO project: board `esp32dev`, framework Arduino.
4. Put this in `platformio.ini`:

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
monitor_speed = 115200
build_flags = -DSIM_MODE=0
lib_deps =
  adafruit/Adafruit SSD1306
  adafruit/Adafruit GFX Library
  bblanchon/ArduinoJson
  knolleary/PubSubClient
```

5. Create `wokwi.toml` in the project root:

```
[wokwi]
version = 1
firmware = '.pio/build/esp32dev/firmware.bin'
elf = '.pio/build/esp32dev/firmware.elf'
rfc2217ServerPort = 4000

# add one block per custom chip (Section 7)
[[chip]]
name = 'pms5003'
binary = 'chips/pms5003.chip.wasm'
```

6. Create `diagram.json`. Easiest route: build the circuit visually at wokwi.com (start from the ESP32 template), then copy its `diagram.json` into your project. Hand-writing wiring JSON is error-prone.
7. Build with PlatformIO, then press F1 and run **Wokwi: Start Simulator**.

## 5. Build and test the circuit in phases

### Phase A: core board

Parts: ESP32 DevKit, SSD1306 OLED, RGB LED with three 220 Ω resistors, relay module, microSD module (if it is in the parts list). Wire as in Figure 1. Write a test sketch that prints `boot ok`, draws text on the OLED, cycles the LED red/green/blue, toggles the relay every 2 s, and writes then reads a file on the SD card.

**Pass:** all five behaviours are visible in the simulator and Serial Monitor. If the SD module is missing, test logging against LittleFS and keep the SD driver for hardware.

### Phase B: stand-in sensors

Add built-in parts: DHT22 in place of the SHT31 (if no SHT31 exists), potentiometers for CO2, TVOC, pressure and air velocity, a PIR for occupancy. Read them through the same functions your final firmware uses.

**Pass:** moving a potentiometer changes the matching value on the OLED and Serial; the PIR toggles `occupied`; all values are in the right units.

### Phase C: faithful sensor emulation

Replace stand-ins one at a time with custom chips (Section 7) so the real UART and I2C drivers run unchanged. Suggested order: PMS5003, MH-Z19B, SGP30, XGZP6847A, LD2410.

**Pass:** the same firmware (`SIM_MODE=0`) reports correct values and copes with protocol errors (bad checksum, no reply).

### Phase D: failure injection

Stop or disconnect a custom chip mid-run. The firmware must mark that sensor `fault`, keep running, and show the fault on the OLED and LED.

## 6. Alternatives for sensors and modules not in Wokwi

Wokwi's parts list changes over time, so verify each part before relying on this table.

| Module | In Wokwi? | Quick stand-in | Faithful option | Software should see |
|---|---|---|---|---|
| MH-Z19B CO2 (UART) | No (verify) | Potentiometer on an ADC1 pin, mapped to 400 to 5000 ppm | Custom UART chip, 9600 baud | co2_ppm |
| PMS5003 PM (UART) | No | Three potentiometers, or fixed values in SIM_MODE | Custom UART chip streaming 32-byte frames | pm1, pm25, pm10 |
| SHT31 temp/RH (I2C 0x44) | Verify, may be missing | DHT22 (built in) | Custom I2C chip | temp_c, rh |

| | | | | |
|---|---|---|---|---|
| SHT31 temp/RH (I2C 0x44) | Verify; may be missing | DHT22 (built in) | Custom I2C chip | temp_c, rh |
| SGP30 TVOC (I2C 0x58) | No | Potentiometer mapped to 0 to 1000 ppb | Custom I2C chip with CRC-8 | tvoc_ppb |
| XGZP6847A pressure (I2C) | No | Potentiometer mapped to your pressure range | Custom I2C chip using datasheet registers | diff_pressure_pa |
| Air velocity (analog) | A potentiometer is an exact electrical stand-in | Potentiometer on ADC1 (include the divider) | Not needed | air_velocity_ms |
| Occupancy, LD2410 (UART) | No | PIR (built in), or HC-SR04 for distance | Custom UART chip, 256000 baud | occupied, distance |
| Occupancy, ToF (I2C) | Verify | HC-SR04 ultrasonic | Custom I2C chip | occupied, distance |
| RGB LED, OLED, relay, microSD | Built in (verify SD) | none needed | none | none |
| Battery, TP4056, buck/boost | Not simulated | Fixed rails in Wokwi | ngspice or LTspice for converters; breadboard test for the charger | battery_v |

## 7. Writing a custom sensor chip

Wokwi custom chips are small C programs compiled to WebAssembly. They appear in `diagram.json` as `chip-<name>` and are registered in `wokwi.toml`. Follow the Chips API docs at docs.wokwi.com for the exact build command and for the syntax of `chip.json` (pins and slider controls). The skeleton below shows the PMS5003 logic; verify the function names against the current docs.

```
// chips/pms5003.chip.c
#include "wokwi-api.h"
#include <stdint.h>
#include <stdlib.h>

typedef struct {
  uart_dev_t uart;
  uint32_t pm1, pm25, pm10;   // values set from chip controls/attributes
} state_t;

static void put16(uint8_t *b, int i, uint16_t v) { b[i] = v >> 8; b[i + 1] = v & 0xFF; }

static void send_frame(void *ud) {
  state_t *s = ud;
  uint8_t f[32] = {0x42, 0x4D, 0x00, 0x1C};      // header + length 28
  uint16_t p1 = attr_read(s->pm1), p25 = attr_read(s->pm25), p10 = attr_read(s->pm10);
  put16(f, 4, p1);  put16(f, 6, p25);  put16(f, 8, p10);     // CF=1
  put16(f, 10, p1); put16(f, 12, p25); put16(f, 14, p10);    // atmospheric
  // bytes 16..29: particle counts + reserved (leave 0 or derive from pm values)
  uint16_t sum = 0;
  for (int i = 0; i < 30; i++) sum += f[i];
  put16(f, 30, sum);                                          // checksum
  uart_write(s->uart, f, 32);
}

void chip_init(void) {
  state_t *s = malloc(sizeof(state_t));
  s->pm1 = attr_init("pm1", 5);
  s->pm25 = attr_init("pm25", 12);
  s->pm10 = attr_init("pm10", 20);
  uart_config_t cfg = {
    .tx = pin_init("TX", INPUT_PULLUP),
    .rx = pin_init("RX", INPUT),
    .baud_rate = 9600,
    .user_data = s,
  };
  s->uart = uart_init(&cfg);
  timer_config_t t = { .callback = send_frame, .user_data = s };
  timer_start(timer_init(&t), 1000000, true);   // one frame per second
}
```

Same pattern for the other sensors:

- **MH-Z19B (UART 9600):** wait for the read command `FF 01 86 00 00 00 00 00 79`, reply with 9 bytes `FF 86 HI LO 00 00 00 00 CS`. ppm = HI × 256 + LO. CS = ((0xFF − sum of bytes 1 to 7) + 1) & 0xFF.
- **SGP30 (I2C 0x58):** accept `0x2003` (init) and `0x2008` (measure). Reply with 6 bytes: eCO2 (2 bytes + CRC), TVOC (2 bytes + CRC). CRC-8 polynomial 0x31, init 0xFF. For the first 15 s after init report 400 ppm and 0 ppb, like the real part.
- **XGZP6847A (I2C):** implement the register map from your datasheet and return a 24-bit raw value scaled by the K factor of your pressure-range variant.
- **LD2410 (UART 256000):** frames start with `F4 F3 F2 F1` and end with `F8 F7 F6 F5`; the payload carries target state (none,

moving, stationary, both) and distances.

## 8. Firmware changes needed for testing

- **Sensor interface with `SIM_MODE`:** 0 = real drivers (used on hardware and with Wokwi chips), 1 = values generated inside the firmware (noise, drift, injected faults) for quick tests.
- **Per-sensor status:** `ok`, `warming_up`, `fault`, `simulated`. Typical warm-up: MH-Z19B about 3 min, PMS5003 about 30 s after the fan starts, SGP30 15 s initial plus long baseline learning.
- **One JSON message** (Section 10) every N seconds with a sequence number, NTP timestamp and a `source` field.
- **Transport:** JSON lines on Serial for Path A, MQTT or HTTP for the other paths.
- **Command handler:** `relay_on`, `relay_off`, `set_interval`, each acknowledged, with a minimum relay on/off time.
- **Local feedback:** OLED page for live values and faults, LED colour for OK, warning, alert and no-network.
- **SD logging** with backfill after the network returns.

```
// sensors.h
struct Reading {
  float co2_ppm, pm1, pm25, pm10, temp_c, rh, tvoc_ppb, dp_pa, air_ms;
  bool occupied;
  uint8_t status[7];   // 0 ok, 1 warming_up, 2 fault, 3 simulated
};
bool readAll(Reading &r);   // implemented by sensors_real.cpp or sensors_sim.cpp (SIM_MODE)
```

## 9. Connecting the simulator to your software

Four paths. Start with A.

**A. Serial bridge (free, local, recommended first).** `wokwi.toml` exposes the ESP32 serial port on localhost:4000. The firmware prints one JSON line per reading, and `bridge.py` forwards it to your backend and writes commands back. This tests the payload format, ingest, AI, dashboard and the relay command path. It does not test the ESP32's WiFi/MQTT code.

```
# bridge.py  (pip install pyserial requests)
import json, requests, serial
ser = serial.serial_for_url('rfc2217://localhost:4000', timeout=1)
while True:
    line = ser.readline().decode(errors='ignore').strip()
    if line.startswith('{'):
        try:
            r = requests.post('http://localhost:8000/api/ingest', json=json.loads(line), timeout=3)
            cmd = r.json().get('command')
            if cmd:
                ser.write((json.dumps(cmd) + '\n').encode())
        except Exception as e:
            print('bridge error:', e)
```

**B. Public MQTT broker.** The simulated ESP32 joins the open virtual network `Wokwi-GUEST` and can reach the internet, so it can publish to a public broker (for example broker.hivemq.com) under a unique topic prefix. Your backend subscribes to the same topics. This tests your real MQTT code. Never send anything sensitive to a public broker.

**C. Tunnel to your local backend.** Expose your backend with a tunnel (ngrok or cloudflared) and have the firmware POST to the HTTPS URL. This tests the HTTP client code.

**D. Wokwi IoT Gateway.** Lets the simulated ESP32 reach your PC's local network, for example a Mosquitto broker on localhost. Availability depends on your Wokwi plan. Note: the port-forwarding option in `wokwi.toml` forwards ports from your PC into the simulated device (for example to a web server running on the ESP32). It does not let the ESP32 reach services on your PC.

## 10. What to add to your software

The examples assume a Python backend such as FastAPI. Adapt them to your stack.

### 10.1 Message contract (share it between firmware and backend)

```
{
  "device_id": "airsafe-001",
  "fw": "0.3.1",
  "seq": 1042,
  "ts": "2026-10-07T12:30:05Z",
  "source": "wokwi",
  "data": {"co2_ppm": 812, "pm1": 6, "pm25": 14, "pm10": 22, "temp_c": 27.4, "rh": 56.2,
           "tvoc_ppb": 130, "diff_pressure_pa": 12.5, "air_velocity_ms": 0.8, "occupied": true},
  "status": {"co2": "ok", "pm": "ok", "th": "ok", "tvoc": "warming_up", "dp": "ok", "vel": "ok", "occ": "ok"},
  "relay": "off"
}
```

MQTT topics: `airsafe/<id>/telemetry`, `airsafe/<id>/status` (retained, with a last-will of offline), `airsafe/<id>/cmd`, `airsafe/<id>/ack`.

### 10.2 Backend checklist

**Ingest and contract**

- HTTP `POST /api/ingest` and/or an MQTT subscriber, both feeding one `process_packet()` function.
- Schema validation against the contract; clear errors for bad packets.
- Dedupe on (device_id, seq); order by timestamp; store server receive time next to device time.
- Device authentication (API key or MQTT credentials).

**Data quality**

- Range checks per sensor (CO2 300 to 10000 ppm, PM2.5 0 to 1000, temperature −40 to 125 °C, RH 0 to 100).
- Stuck-value and spike detection. Honour per-sensor status: exclude `warming_up` and `fault` values from alerts and AI.
- Staleness: device is stale after 2 missed intervals, offline after more.
- A `source` field (real, wokwi, simulated) on every record, shown on the dashboard so test data never mixes with real data.

**Storage**

- Raw packets plus cleaned values (SQLite is fine for testing, a time-series store later) and a retention policy.

**Analysis and AI**

- Feature pipeline (rolling mean, rate of change, time of day), AQI or comfort index, anomaly detection, ventilation or risk recommendation.
- Golden tests: fixed input series with known expected outputs. Define what happens when the model errors or data is missing, and measure latency.

**Alerts and control**

- Threshold rules with hysteresis (for example fan on above 1000 ppm CO2, off below 800) and minimum on/off times.
- A command queue per device: send, wait for ack, retry, time out, and show state (pending, applied, failed). Manual override from the dashboard.

**Dashboard**

- Live values, per-sensor health badges, device online state, relay state, alert history and a clear SIMULATED/REAL banner.

**Test tooling**

- A sensor simulator (10.4) that plays named scenarios.
- pytest end-to-end tests that start the backend, play a scenario, and assert alerts, relay commands and stored rows.
- One config switch, for example `DATA_SOURCE=simulated|wokwi|real`.

### 10.3 FastAPI ingest skeleton

```
from datetime import datetime
from fastapi import FastAPI
from pydantic import BaseModel, Field

app = FastAPI()
pending_cmd = {}          # device_id -> command dict, set by your rules engine

class Data(BaseModel):
    co2_ppm: float | None = Field(None, ge=300, le=10000)
    pm25: float | None = Field(None, ge=0, le=1000)
    temp_c: float | None = Field(None, ge=-40, le=125)
    rh: float | None = Field(None, ge=0, le=100)
    # add pm1, pm10, tvoc_ppb, diff_pressure_pa, air_velocity_ms, occupied

class Packet(BaseModel):
    device_id: str
    seq: int
    ts: datetime
    data: Data
    status: dict[str, str] = {}
    source: str = 'real'

@app.post('/api/ingest')
def ingest(p: Packet):
    # 1 dedupe  2 store raw  3 clean + analyse  4 raise alerts  5 decide relay
    return {'ok': True, 'command': pending_cmd.pop(p.device_id, None)}
```

### 10.4 Sensor simulator

```
# simulate.py: plays scenarios against your backend
import random, time, requests
URL = 'http://localhost:8000/api/ingest'

def closed_room(m):   # m = simulated minutes; CO2 climbs, PM steady
    return dict(co2_ppm=450 + 60 * m + random.gauss(0, 8), pm25=12 + random.gauss(0, 2),
                temp_c=26 + 0.05 * m, rh=55, tvoc_ppb=100 + 5 * m,
                diff_pressure_pa=12, air_velocity_ms=0.5, occupied=True)

def run(scenario, minutes=30, speedup=60):
    base = time.time()
    for seq in range(minutes):
        ts = time.strftime('%Y-%m-%dT%H:%M:%SZ', time.gmtime(base + seq * 60))
        pkt = dict(device_id='sim-001', seq=seq, ts=ts, data=scenario(seq), status={}, source='simulated')
        print(requests.post(URL, json=pkt, timeout=3).json())
        time.sleep(60 / speedup)

run(closed_room)
```

Add one function per scenario in Section 11.

## 11. End-to-end test plan

| # | Scenario | Inject | Expected software behaviour |
|---|---|---|---|
| 1 | Normal day | Baseline values with small noise | No alerts, plausible charts |
| 2 | Closed occupied room | CO2 450 to 1800 ppm over 20 simulated min | Alert at threshold, relay ON command, relay OFF after CO2 falls below the lower threshold |
| 3 | PM spike (cooking, incense) | PM2.5 12 to 250, then decay | AQI category changes, alert, recovery |
| 4 | Sensor warm-up | `warming_up` status for the first minutes | Values stored but excluded from alerts and AI |
| 5 | Stuck sensor | Same value for 10+ readings | Flagged suspect |
| 6 | Out-of-range value | CO2 65535 or temperature −999 | Rejected, counted, sensor shown as fault |
| 7 | Dropout | No packets for 2+ intervals | Device stale, then offline |
| 8 | Duplicate and out-of-order packets | Repeat `seq`, send older timestamps | Deduped, ordered by timestamp |
| 9 | Noisy data with spikes | Gaussian noise plus single-sample spikes | Filtering works, no false alerts |
| 10 | Lost relay command | Command sent, no ack | Retry, then shown as failed |
| 11 | Backend down | Stop the server for 5 min | Device buffers or drops as designed, backfills after reconnect |
| 12 | Wrong device clock | Bad timestamp | Server time used, skew flagged |

An end-to-end pass means: every scenario produces the expected rows, alerts and commands with no unhandled exceptions, and

the dashboard matches the stored data.

## 12. Moving to real hardware

**Bring-up order**

1. Power section alone with a dummy load; measure the 5 V and 3.3 V rails, including on battery.
2. ESP32 + OLED; run an I2C scan.
3. Add I2C sensors one at a time and compare each against the datasheet and your simulated values.
4. Add UART sensors (PMS5003, MH-Z19B, occupancy), each with its own test.
5. Relay with a low-voltage test load first.
6. Run a 24 hour soak test and check dropouts, resets and drift.

**What simulation cannot prove**

- Sensor accuracy, noise, warm-up behaviour and calibration (CO2 baseline, SGP30 baseline).
- Power dips when the PMS5003 fan starts, brownouts, battery runtime and charger behaviour.
- I2C and UART reliability on real wires, EMI, and WiFi range.
- Airflow effects on the particle, pressure and velocity sensors, and thermal self-heating of the temperature sensor.
- Mains safety of the AC-DC stage and relay wiring.